Formal Verification of the Brevis Pico RISC-V zkVM

Draft Technical Report



NETHERMIND

7th of April, 2026

1 Executive Summary

This report summarises the engagement performed by Nethermind on formally verifying, in the Lean proof assistant, the Brevis Pico RISC-V zkVM implementation of the RV32IM instruction set against the Lean RISC-V specification.

The work done in this engagement was based on the the official Brevis Pico repository (commit `3b074f0`, dating from February 15th, 2026), and the engagement will resulted in the following three deliverables:

1. a mechanism for extracting Plonky3 constraints from Pico chips, to be made publicly available;
2. a comprehensive verification infrastructure written in the Lean proof assistant (version 4.28.0-rc1), available at `github.com/NethermindEth/pico-fv`; and
3. as part of this infrastructure, Lean proofs that the Pico zk implementations of the following 45 RISC-V RV32IM opcodes satisfy their Lean RISC-V specifications:
   - 27 ALU opcodes: ADD, ADDI, SUB, XOR, XORI, OR, ORI, AND, ANDI, SLT, SLTI, SLTU, SLTUI, SLL, SLLI, SRL, SRLI, SRA, SRAI, MUL, MULH, MULHU, MULHSU, DIV, DIVU, REM, and REMU;
   - 10 control-flow opcodes: AUIPC, BEQ, BNE, BLT, BGE, BLTU, BGEU, LUI, JAL, and JALR; and
   - 8 memory manipulation opcodes: LW, LH, LHU, LB, LBU, SW, SH, and SB.

Running the proofs. Assuming that an installation of Lean's lake package manager is present on the system, the infrastructure and all of the proofs can be built using:

```
lake update
lake exe cache get!
lake build
```

in the root folder of the repository, where the first two lines are superfluous but allow for a faster initial build.

1.1 Verification context, assumptions, and caveats

The verification was performed in the context of future participation of Pico in Ethereum L1 proving, for which the Ethereum Foundation (EF) is developing a proposal for the standardisation of the RISC-V target architecture for Ethereum with the goal of defining a minimal RISC-V instruction set for zkEVMs that would like to prove Ethereum blocks. This proposal, at the time of writing this report, makes the following recommendations that are relevant for this engagement:

EF1: that the supported set of instructions should be RV64IM: Pico currently supports RV32IM, which is the instruction set that we have verified in full as part of this engagement;

EF2: that the only supported privileged mode should be machine mode (M);

EF3: that no interaction with the environment using system calls should be required;

EF4: that the memory should be treated as flat, without advanced use of the memory management unit (MMU) and related concepts, such as paging; and

EF5: that misaligned memory accesses should be allowed: Pico currently supports only aligned memory accesses.

On the other hand, the Lean RISC-V specification models almost all of the mechanisms, both mandatory and optional, described by the RISC-V standard and various extensions. In line with EF recommendations and the design choices made by Pico developers, we modify the 32-bit and 64-bit variants of the Lean RISC-V specification as follows:

S1: We disable the Zca RISC-V extension by setting its support to `false` here (32-bit) and here (64-bit). The Zca extension enables the RISC-V C instruction set, which allows for higher data compression. This extension is not implemented by Pico and is not required for L1 proving.

S2: We disable the Zicfilp RISC-V extension by setting its support to `false` here (32-bit) and here (64-bit). The Zicfilp extension enforces forward-edge control-flow integrity by introducing a landing-pad instruction, LPAD, which must be placed at the program locations that are valid targets of indirect jumps or calls. Pico does not support the LPAD instruction, and there is no indication that this support is required for L1 proving. In the proofs, this is relevant only for the JALR opcode.

S3: We disable the RISC-V core-local interruptor (CLINT), which is a memory-mapped peripheral that handles software and timer interrupts, by setting its size to zero here (32-bit) and here (64-bit). This mechanism is not required for L1 proving, and disabling it is also in line with recommendation EF4 and the uniform way in which Pico models memory. In the proofs, this is relevant for all of the memory-manipulating opcodes.

S4: We disable the RISC-V Physical Memory Protection (PMP) mechanism here (32-bit) and here (64-bit). The PMP is not modelled by Pico, and disabling it is in line with recommendation EF4. In the proofs, this is relevant for all of the memory-manipulating opcodes.

Further, we make the following assumptions on the RISC-V state in our Lean development, formalised here (32-bit) and here (64-bit), and given in full for the 32-bit version below:

A1: We 1) set the privileged mode to be machine mode; and 2) do not allow for this privilege to be overridden by setting the `MPRV` bit of the `mstatus` register to zero. This is in line with EF2.

A2: We set up the Physical Memory Attributes (PMA) in line with EF2. Specifically:

1. We declare there to be a single PMA region. This is to make sure that all correctly aligned memory accesses will be executed, disallowing cases in which the load or store would cross beyond the end of a PMA region (for example, if the size of a PMA region is not a multiple of four, and a four-byte load was requested from the address that is the multiple of four closest to its end).
2. We set the base address of this single PMA region to be zero, and its size to be at least 2130706440 (that is, $\texttt{KoalaBearPrime} + 3$) bytes for the 32-bit version. In doing so, we ensure that the actual address space of 32-bit Pico is supported (for more details, see finding 1 below).
3. We set this entire single PMA region to be readable, writable, and with misaligned accesses allowed, in line with the EF zkVM standard recommendations. We note, however, that Pico at the time of writing only supported aligned memory accesses.

A3: We disable the host/target mechanism of RISC-V. This is in line with EF3, as support for this mechanism should not be required for L1 proving.

A4: In order for all opcodes to be executable by the Lean RISC-V specification, we assume the existence of: 1) the `misa` register, required for most control-flow opcodes; and 2) the `mseccfg` register, required for JALR.

```
-- Pico 32-bit address space size
notation "Pico_address_space_size_64" ⇒ 2130706440

-- RISC-V assumptions, 32-bit variant
def RISC_V_assumptions
  (state : PreSail.SequentialState RegisterType Sail.trivialChoiceSource)
  (mstatus : RegisterType Register.mstatus)
  (pmaRegion : PMA_Region)
  (misa : RegisterType Register.misa)
  (mseccfg : RegisterType Register.mseccfg)
: Prop :=
  -- Assumption A1.1: machine privilege
  Sail.readReg Register.cur_privilege state = EStateM.Result.ok Privilege.Machine state ∧
  -- Assumption A1.2: MPRV bit of the mstatus register not set
  (Sail.readReg Register.mstatus state = EStateM.Result.ok mstatus state ∧
   BitVec.extractLsb 17 17 mstatus = 0#1) ∧
  -- A2.1 : Single PMA region
  Sail.readReg Register.pma_regions state = EStateM.Result.ok [ pmaRegion ] state ∧
  -- A2.2 : ...with base 0 and at least 2^29 bytes in size...
  pmaRegion.base = 0 ∧ Pico_address_space_size_32 ≤ pmaRegion.size.toNat ∧
  -- A2.3 : ...with all addresses readable and writable, and misaligned accesses are allowed
  pmaRegion.attributes.readable ∧ pmaRegion.attributes.writable ∧
  pmaRegion.attributes.misaligned_fault = misaligned_fault.NoFault ∧
  -- Assumption A3: no host-target interface
  Sail.readReg Register.htif_tohost_base state = EStateM.Result.ok .none state ∧
  -- Assumption A4.1: misa register exists
  Sail.readReg Register.misa state = EStateM.Result.ok misa state ∧
  -- Assumption A4.2: mseccfg register exists
  Sail.readReg Register.mseccfg state = EStateM.Result.ok mseccfg state
```

Finally, when it comes to the underlying infrastructure, we assume the following:

I1: correctness of the theory and implementation of the Pico lookup argument and bus interaction mechanism;

I2: correctness of the theory and implementation of the proof system utilised by Pico;

I3: correctness of the RISC-V Lean specification with respect to the official RISC-V standard; and

I4: correctness of Lean's kernel.

1.2 Summary of findings

All of the relevant findings are related to either a general property of the program counter or to the `MemoryReadWrite` air. The other airs have been implemented fully correctly, following the RISC-V specification.

1. The program counter must be enforced during the transpilation phase to have an upper bound that does not allow overflow (e.g., 2^{30}).
2. Currently, what is declared to be the maximal memory address is set to be $\leq$ `BabyBearPrime`. This setting, however, allows one to have an aligned memory address of `BabyBearPrime` $- 1$, which means that the effective largest memory address accessible to a read or write is `BabyBearPrime` $+ 2$. This can be corrected in two ways:
   (a) the maximal memory address could be constrained to be strictly less than a specified limit that must be divisible by four; or
   (b) the documentation should be adapted to correctly reflect the actual maximal memory address.
3. Given the design of the `State` bus, which captures both register writes (< 32) and memory writes (≥ 32), in order to ensure that the meaning of the `Memory` bus is aligned with the intended meaning of the `State` bus, the memory address read from or written to must explicitly be constrained to be greater than or equal to 32 in the constraints. Otherwise, one could be attempting to write into memory but instead be writing to a register. This also means that memory at addresses from 0-31 is fully inaccessible.
4. The computation of `mem_value_is_neg_not_x0` (here) and `mem_value_is_pos_not_x0` (here) must not be conditional on `op_a_0`. Otherwise, when the opcode is a load and the register to be written to is zero, the `Memory` bus does not have the intended meaning required by the RISC-V spec which is that its a-values must denote the read value regardless of the register to be written to.

2 Verification methodology

We discuss the key aspects of our methodology: interfacing with the RISC-V specification, the treatment of buses (both general and Pico-specific), formalisation of transpilation, and the workflow of per-opcode verification.

2.1 Interface with the RISC-V specification

We identify the six basic operations that all RV32IM opcodes perform on the RISC-V state:

```
-- Width of supported memory accesses
inductive MemoryWidth where
| Byte | Half | Word

-- Width of supported memory accesses as a natural number
def width_of_mem_width (mw : MemoryWidth) :=
  match mw with | .Byte ⇒ 8 | .Half ⇒ 16 | .Word ⇒ 32

-- Relevant RISC-V state operations
inductive Operation where
| ReadPC   (val : BitVec 32) : Operation -- Read value of program counter
| WritePC  (val : BitVec 32) : Operation -- Write value of next program counter
-- Read/write value of given register
| ReadReg  (reg : regidx) (val : BitVec 32) : Operation
| WriteReg (reg : regidx) (val : BitVec 32) : Operation
-- Read/write memory at given address and specified width
| ReadMem  (addr : ℕ) (mw : RV64MemoryWidth) (val : BitVec (width_of_mem_width mw)) : Operation
| WriteMem (addr : ℕ) (mw : RV64MemoryWidth) (val : BitVec (width_of_mem_width mw)) : Operation
```

Next, we capture the effect of these operations (eliding lengthy non-informative types) as follows:

```
structure OperationEffect where
  hyps : Prop
  state : EStateM.Result (...) (...) ExecutionResult
```

where hyps denotes the facts about the state we learn from read-operations, and `state` denotes the state obtained from a given state using write-operations.

Next, and most importantly, we formalise the effect of a single operation on a given RISC-V state, giving the implementation below (eliding some repetitions and technicalities for clarity) and providing explanation in comments:

```
def operation_effect
  (result : OperationEffect)
  (entry : Operation)
: OperationEffect :=
  match result.state with
  -- Previous operation raised an error, propagate last state
  | .error _ _ ⇒ result
  -- Previous operation was successful
  | .ok _ state ⇒
    match entry with
    -- Reading the program counter learns its current value and does not modify the state
    | .ReadPC val ⇒
      { result with hyps := result.hyps ∧ Sail.readReg Register.PC state = EStateM.Result.ok val state }
    -- Writing the program counter learns nothing and performs the appropriate update
    | .WritePC val ⇒
      { result with state := EStateM.Result.map (...) (Sail.writeReg Register.nextPC val state) }
    -- Reading a register learns its current value and does not modify the state
    | .ReadReg reg val ⇒
      { result with hyps := result.hyps ∧ read_xreg (regidx_to_fin reg) state = EStateM.Result.ok val state }
    -- Writing a register learns nothing and performs the appropriate update
    | .WriteReg reg val ⇒
      { result with state := EStateM.Result.map (...) (write_xreg (regidx_to_fin reg) val state) }
    | .ReadMem addr mw val ⇒
      -- Loading a single byte learns the value of that byte in memory
```

```
      match mw with
      | .Byte ⇒ { result with hyps := result.hyps ∧ state.mem[addr]? = .some val }
      | .Half ⇒ ... -- analogous to Word below
      -- Loading a word value learns the value of the four bytes that comprise it
      | .Word ⇒
        let data0 := BitVec.extractLsb  7  0 val; let data1 := BitVec.extractLsb 15  8 val
        let data2 := BitVec.extractLsb 23 16 val; let data3 := BitVec.extractLsb 31 24 val
        { result with hyps := result.hyps ∧
                              state.mem[addr]?     = .some data0 ∧ state.mem[addr + 1]? = .some data1 ∧
                              state.mem[addr + 2]? = .some data2 ∧ state.mem[addr + 3]? = .some data3}
    | .WriteMem addr mw val ⇒
      { result with
          state :=
            match mw with
            | .Byte ⇒ EStateM.Result.ok (...) { state with mem := state.mem.insert addr val }
            | .Half ⇒ ... -- analogous to Word below
            -- Writing a word learns nothing and writes the four bytes that the value represents
            | .Word ⇒
              let data0 := BitVec.extractLsb  7  0 val; let data1 := BitVec.extractLsb 15  8 val
              let data2 := BitVec.extractLsb 23 16 val; let data3 := BitVec.extractLsb 31 24 val
              EStateM.Result.ok (...) {
                state with
                  mem := (((state.mem.insert addr data0).insert (addr + 1) data1
                          ).insert (addr + 2) data2).insert (addr + 3) data3
              }
      }
```

Next, we formalise an effect of a sequence of operations on an initial RISC-V state, returning the hypotheses obtained from the reads and the state obtained after the writes:

```
def operation_effect
  (ops : List Operation)
  (state : PreSail.SequentialState RegisterType Sail.trivialChoiceSource)
: OperationEffect :=
  let initial_result := { hyps := True, state := EStateM.Result.ok (ExecutionResult.Retire_Success ()) state }
  List.foldl operation_effect initial ops
```

Finally, for each RV32IM opcode, we give a sequence of operations that we prove equivalent to executing the opcode using the RISC-V semantics. We give examples of ADD, BNE, JALR, LB, and SW below with explanations inlined:

```
def spec_ADD
  (r1 r2 rd : regidx)
  (pc_val r1_val r2_val : BitVec 32)
: List Operation :=
  [
    .ReadPC pc_val,                 -- Read the current `pc`
    .ReadReg r1 r1_val,             -- Read register `r1`, obtaining `r1_val`
    .ReadReg r2 r2_val,             -- Read register `r2`, obtaining `r2_val`
    .WritePC (pc_val + 4#32),       -- Write the next `pc` (current `pc` + 4)
    .WriteReg rd (r1_val + r2_val), -- Write the sum to register `rd`^^I
  ]

def spec_BNE
  (r1 r2 : regidx) (imm : BitVec 13)
  (pc_val r1_val r2_val : BitVec 32)
: List Operation :=
  -- The `pc` increment is either 4 (if `r1_val = r2_val`) or sign-extended immediate `imm` (otherwise)
  let increment := if r1_val == r2_val then 4#32 else BitVec.signExtend 32 imm
  [
    .ReadPC pc_val,                -- Read the current `pc`
    .ReadReg r1 r1_val,            -- Read register `r1`, obtaining `r1_val`
    .ReadReg r2 r2_val,            -- Read register `r2`, obtaining `r2_val`
    .WritePC (pc_val + increment), -- Write the next `pc`
  ]
```

```
def spec_JALR
  (r1 rd : regidx) (imm : BitVec 12)
  (pc_val r1_val : BitVec 32)
: List Operation :=
  let mask := 0xFFFFFFFE
  [
    .ReadPC pc_val,                                          -- Read the current `pc`
    .ReadReg r1 r1_val,                                      -- Read register `r1`, obtaining `r1_val`
    .WritePC (mask &&& (rs1_val + BitVec.signExtend 32 imm)), -- Write the next `pc`
    .WriteReg rd (pc_val + 4#32),                            -- Write the return `pc` to register `rd`
  ]

def spec_LB
  (r1 rd : regidx) (imm : BitVec 12)
  (pc_val r1_val : BitVec 32)
  (data0 : BitVec 8)
: List Operation :=
  -- The targeted address is the sum of the value of register `r1` and the sign-extended immediate `imm`
  let addr := (r1_val + BitVec.signExtend 32 imm).toNat
  [
    .ReadPC pc_val,                          -- Read the current `pc`
    .ReadReg r1 r1_val,                      -- Read register `r1`, obtaining `r1_val`
    .ReadMem addr .Byte data0,               -- Read required byte from the desired memory address
    .WritePC (pc_val + 4#32),                -- Write the next `pc`
    .WriteReg rd (BitVec.signExtend 32 data0), -- Write read value, 32-bit-sign-extended, to register `rd`
  ]

def spec_SW
  (r1 r2 : regidx) (imm : BitVec 12)
  (pc_val r1_val r2_val : BitVec 32)
: List Operation :=
  -- The targeted address is the sum of the value of register `r1` and the sign-extended immediate `imm`
  let addr := (r1_val + BitVec.signExtend 32 imm).toNat
  [
    .ReadPC pc_val,              -- Read the current `pc`
    .ReadReg r1 r1_val,          -- Read register `r1`, obtaining `r1_val`
    .ReadReg r2 r2_val,          -- Read register `r2`, obtaining `r2_val`
    .WritePC (pc_val + 4#32),    -- Write the next `pc`
    .WriteMem addr .Word r2_val, -- Write `r2_val` to the desired memory address
  ]
```

The equivalence lemmas are then of the form (given below for BNE):

```
lemma execute_ADD_equiv
  (h : (operation_list_effect (spec_ADD r1 r2 rd pc_val r1_val r2_val) state).hyps)
:
  execute_instruction (instruction.RTYPE (r2, r1, rd, .ADD)) state =
  (operation_list_effect (spec_ADD r1 r2 rd pc_val r1_val r2_val) state).state
```

where `execute_instruction` executes the RISC-V instruction at hand using the RISC-V spec:

```
noncomputable def execute_instruction
  (instr : instruction)
  (state : PreSail.SequentialState RegisterType Sail.trivialChoiceSource)
:=
  (do
    Sail.writeReg Register.nextPC (Sail.BitVec.addInt (← Sail.readReg Register.PC) 4)
    LeanRV32D.Functions.execute instr
  ) state
```

With these lemmas in place, we have effectively simplified the burden of correctness proofs to only having to show that the implementation of a given opcode performs a sequence of operations on the RISC-V state that has the same effect as the canonical operations of that opcode.

2.2 General treatment of buses

Pico uses a number of buses for inter-chip communication and LogUp-based lookup-table information retrieval. A general bus entry over a field `F` is, in principle, of the form:

```
⟨multiplicity : F, data : Vector F⟩
```

consisting of a multiplicity (most commonly equalling 0, 1, or -1) and a vector of data, which can have different lengths and carry different meaning for different buses.

We formalise the notion of a bus as a Lean class, as follows:

```
/-- Type class for generic bus entries -/
class BusEntry (α : Type*) where
  multiplicity : α → F

  data_length : ℕ
  data : α → Vector F data_length

  -- Infrastructure-external constraints on bus entries
  axioms : α → Prop

  -- Chip-external, but infrastructure-internal constraints on bus entries
  externals : α → Prop

  -- Well-formedness properties of bus entries
  wf_properties : α → Prop

  -- External properties assumed on write, asserted on read
  ext_assume_cond := fun entry ⇒ entry.1 = 1,
  ext_assert_cond := fun entry ⇒ entry.1 = -1,

  -- WF-properties assumed on read, asserted on write
  wf_assume_cond := fun entry ⇒ entry.1 = -1,
  wf_assert_cond := fun entry ⇒ entry.1 = 1,

  -- Assumption and assertion of external properties
  ext_assume (a : α) := ext_assume_cond a → externals a
  ext_assert (a : α) := ext_assert_cond a → externals a

  -- Assumption and assertion of WF-properties
  wf_assume (a : α) := wf_assume_cond a → wf_properties a
  wf_assert (a : α) := wf_assert_cond a → wf_properties a

  -- Serialisation, deserialisation, inverses
  serialise (x : α) := (multiplicity x, data x)
  serialiseToList (x : α) := (multiplicity x, (data x).toList)
  deserialise : F × Vector F data_length → α

  inv_ser_deser (x : F × Vector F data_length) : serialise (deserialise x) = x
  inv_deser_ser {x : α} : deserialise (serialise x) = x
```

providing the ability to specify:

- the bus entry multiplicity, the length of the data, and the data itself;
- the bus entry axioms, i.e., the conditions enforced by the Pico implementation that are not part of any air;
- the bus entry external properties, i.e., the conditions that are not enforced by airs emitting the entry but are enforced by the calling airs;
- the bus entry well-formedness (WF) properties, i.e., the conditions that are assumed to hold for every bus entry an air reads and must be proven to hold for every bus entry an air writes;
- the conditions under which external properties and WF-properties should be assumed and asserted:

  - when it comes to external properties: if an air reads a bus entry, then in must satisfy the required external properties to read it (assert); and if an air writes a bus entry, then it can assume to have the required external properties to write it; and
  - when it comes to wf-properties: if an air reads a bus entry, then this bus entry must have been well-formed (assume), and if an air writes a bus entry, then this entry must be well-formed (assert).
- how to generically serialise a bus entry into a tuple and a list; and
- how to deserialise a bus entry correctly.

2.3 The Pico buses

Pico effectively uses five buses in the implementation of the RISC-V opcodes relevant for this engagement. We discuss each of them in the upcoming subsections. The important point to remember is that the semantics of the bus entries is effectively given by the axioms, external properties, well-formedness properties, and their effect on the RISC-V state (introduced shortly).

2.3.1 The `Program` bus and the RISC-V Instruction Transpiler

The way in which Pico connects to the RISC-V execution is by transpiling the RISC-V instructions to be executed to appropriate `Program` bus entries, which have the following structure:

```
structure ProgramBusEntry where
  -- lookup multiplicity
  multiplicity : F
  -- program counter
  pc : F
  -- instruction opcode
  opcode opcode' : F
  -- a-operand/register
  a_0 a_1 a_2 a_3 : F
  -- b-operand/register
  b_0 b_1 b_2 b_3 : F
  -- c-operand/register
  c_0 c_1 c_2 c_3 : F
  -- Indicator if the write register is the zero register
  a_is_0 : F
  -- Immediate indicators
  imm_b imm_c : F
  -- Instruction selectors
  is_alu : F
  is_ecall : F
  is_lb is_lbu is_lh is_lhu is_lw : F
  is_sb is_sh is_sw : F
  is_beq is_bne is_blt is_bge is_bltu is_bgeu : F
  is_jalr is_jal : F
  is_auipc : F
  is_unimpl : F
```

We formalise this transpiler in full following its implementation in the Pico codebase. One interesting detail is that the LUI opcode does not have a dedicated corresponding opcode, but is instead transpiled as an addition, given that its argument already comes correctly shifted in the instruction format. In more detail, the transpilation function is defined as follows:

```
  def transpile_op
    (inst : instruction) -- RISC-V instruction
    (multiplicity : FKB) -- Program bus entry multiplicity
    (pc : FKB)           -- program counter
    -- Returns:
    -- 1. a deserialised program bus entry; and
    -- 2. opcode-related facts guaranteed by the implementation.
  : Option ((FKB × Vector FKB 38) × Prop) :=
    ...
```

and, for example, the transpilation of the six branching opcodes is handled as follows:

```
-- BTYPE : 6 opcodes
| .BTYPE (imm, rs2, rs1, op) ⇒
  let opcode := opcode_of_bop op
  let a_is_0 := 0
  let imm_KB := imm_to_KB imm true
  .some (
    (
      multiplicity,
      #v[
        pc, opcode, opcode,
        reg_to_KB rs1, 0, 0, 0,
        reg_to_KB rs2, 0, 0, 0,
        imm_KB[0], imm_KB[1], imm_KB[2], imm_KB[3],
        a_is_0,
        0, 1,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        (if op == .BEQ then 1 else 0),
        (if op == .BNE then 1 else 0),
        (if op == .BLT then 1 else 0),
        (if op == .BGE then 1 else 0),
        (if op == .BLTU then 1 else 0),
        (if op == .BGEU then 1 else 0),
        0, 0, 0, 0
      ]
    ),
    is_register (reg_to_KB rs1) ∧
    is_register (reg_to_KB rs2) ∧
    imm % 2 = 0
  )
```

```
def imm_to_KB {w : ℕ}
  (imm : BitVec w) (sgn : Bool)
: Vector FKB 4 :=
  let imm_u32 :=
    U32.fromNat (BitVec.extend imm 32 sgn).toNat
  Vector.map (fun x ⇒ ⟨ x.toNat, by omega ⟩) imm_u32
```

```
def opcode_of_bop (op : bop) : FKB :=
  match op with
  | .BEQ  ⇒ 18 | .BNE  ⇒ 19
  | .BLT  ⇒ 20 | .BGE  ⇒ 21
  | .BLTU ⇒ 22 | .BGEU ⇒ 23
```

```
def is_register (reg : FKB) : Prop :=
  reg < 32
```

where we see that the additional facts guarantee that `rs1` and `rs2` are registers (that is, less than 32) and that the immediate, which denotes the relative jump, is divisible by two, a property of the instruction format.

The program bus instance is then formalised as

```
/-- Program bus entry instance -/
instance ProgramBusEntryInstance
: BusEntry FKB (ProgramBusEntry FKB) :=
{
  multiplicity := fun entry ⇒ entry.1,

  data_length := 38,
  data := fun ⟨_, pc, opcode, opcode', a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3, c_0, c_1, c_2, c_3, a_is_0,
              imm_b, imm_c, is_alu, is_ecall, is_lb, is_lbu, is_lh, is_lhu, is_lw, is_sb, is_sh, is_sw,
              is_beq, is_bne, is_blt, is_bge, is_bltu, is_bgeu, is_jalr, is_jal, is_auipc, is_unimpl⟩ ⇒
            #v[pc, opcode, opcode', a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3, c_0, c_1, c_2, c_3, a_is_0,
              imm_b, imm_c, is_alu, is_ecall, is_lb, is_lbu, is_lh, is_lhu, is_lw, is_sb, is_sh, is_sw,
              is_beq, is_bne, is_blt, is_bge, is_bltu, is_bgeu, is_jalr, is_jal, is_auipc, is_unimpl]

  -- Axioms : program counter is bounded to 2^30 and aligned to a multiple of four
  axioms := fun entry ⇒
   (¬ entry.multiplicity = 0) → entry.pc < 2^30 ∧ entry.pc % 4 = 0

  -- No chip-external constraints
  externals := fun _ ⇒ True

  -- Well formedness derived from the fact that the entries come from the transpiler
  wf_properties := ProgramBusEntry.operand_properties

  ...
}
```

where operand properties are defined through transpilation as follows:

```
def ProgramBusEntry.operand_properties (entry : ProgramBusEntry FKB) : Prop :=
  ∃ instruction data props,
    (.some ⟨ data, props ⟩ = Transpiler.transpile_op instruction entry.multiplicity entry.pc) ∧
    (entry = ProgramBusEntry.deserialise data) ∧
    props
```

With respect to this formalisation, we note that the axiomatic characterisation of the program counter is enforced by the transpiler code: the upper bound (to be implemented, cf. §1.2, Finding 1) is required to prevent overflow in the finite field, and the alignment needs to be enforced (here and here for the `Program` bus) as the RISC-V compressed-instruction "C" extension is not supported by Pico. The above code enforces that the program counters of the CPU air are divisible by four, whereas the proof that the final next program counters of the CPU air is divisible by four requires reasoning about the recursion air that is out of scope of this formalisation, but still holds and is captured with an assumption and elaborated on here.

2.3.2 The ALU bus

The ALU bus captures the effect of the ALU-related opcodes. Its entry structure and semantics are as follows:

```
/-- ALU bus entry -/
structure ALUBusEntry where
  multiplicity : F -- lookup multiplicity
  ALU_opcode   : F -- ALU opcode
  -- operands: a = b <<op>> c
  a_0 a_1 a_2 a_3 : F
  b_0 b_1 b_2 b_3 : F
  c_0 c_1 c_2 c_3 : F

/-- ALU bus entry instance -/
instance ALUBusEntryInstance : BusEntry FKB (ALUBusEntry FKB) :=
{
  multiplicity := fun entry ⇒ entry.1,

  data_length := 13,
  data := fun ⟨_, ALU_opcode, a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3, c_0, c_1, c_2, c_3⟩ ⇒
            #v[ALU_opcode, a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3, c_0, c_1, c_2, c_3]

  -- No axioms
  axioms := fun _ ⇒ True

  -- Externals
  externals := fun entry ⇒
    ¬(entry.multiplicity = 0) →
      -- The ALU opcode range
      (entry.ALU_opcode ∈ Finset.Icc 0 9 ∨ entry.ALU_opcode ∈ Finset.Icc 30 37) ∧
      -- Operands are range-checked externally to the chip for most ALU operations
      (entry.ALU_opcode > 4 →
         entry.b_0 < 256 ∧ entry.b_1 < 256 ∧ entry.b_2 < 256 ∧ entry.b_3 < 256 ∧
         entry.c_0 < 256 ∧ entry.c_1 < 256 ∧ entry.c_2 < 256 ∧ entry.c_3 < 256)

  -- Well-formedness captured by pure specification
  wf_properties :=
    fun ⟨ _, ALU_opcode, a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3, c_0, c_1, c_2, c_3 ⟩ ⇒
      ALUBus.Spec ALU_opcode a_0 a_1 a_2 a_3 b_0 b_1 b_2 b_3 c_0 c_1 c_2 c_3

  ...
}
```

where the specification of what each ALU opcode does, `ALUBus.Spec`, is defined as follows:

```
/-- ALU bus pure specification -/
def ALUBus.Spec (ALU_opcode a_0 a_1 a_2 a_3 b_0 b_1 b_2 b_3 c_0 c_1 c_2 c_3 : FKB) :=
  let result_range  := a_0 < 256 ∧ a_1 < 256 ∧ a_2 < 256 ∧ a_3 < 256
  let operand_range := b_0 < 256 ∧ b_1 < 256 ∧ b_2 < 256 ∧ b_3 < 256 ∧
```

```
                    c_0 < 256 ∧ c_1 < 256 ∧ c_2 < 256 ∧ c_3 < 256
  let a := U32.toBV #v[a_0, a_1, a_2, a_3]
  let b := U32.toBV #v[b_0, b_1, b_2, b_3]
  let c := U32.toBV #v[c_0, c_1, c_2, c_3]
    result_range ∧
    (if ALU_opcode < 5 then operand_range else True) ∧
    match ALU_opcode with
    | 0  ⇒ a = b + c
    | 1  ⇒ a = b - c
    | 2  ⇒ a = b ^^^ c
    | 3  ⇒ a = b ||| c
    | 4  ⇒ a = b &&& c
    | 5  ⇒ a = b <<< (c.toNat % 32)
    | 6  ⇒ a = b >>> (c.toNat % 32)
    | 7  ⇒ a = b.sshiftRight (c.toNat % 32)
    | 8  ⇒ a = if b.toInt < c.toInt then 1#32 else 0#32
    | 9  ⇒ a = if b.toNat < c.toNat then 1#32 else 0#32
    | 30 ⇒ a = RV32IM.execute_MUL_pure b c .MUL
    | 31 ⇒ a = RV32IM.execute_MUL_pure b c .MULH
    | 32 ⇒ a = RV32IM.execute_MUL_pure b c .MULHU
    | 33 ⇒ a = RV32IM.execute_MUL_pure b c .MULHSU
    | 34 ⇒ a = (RV32IM.execute_DIV_REM_pure b c .DRS).1
    | 35 ⇒ a = (RV32IM.execute_DIV_REM_pure b c .DRU).1
    | 36 ⇒ a = (RV32IM.execute_DIV_REM_pure b c .DRS).2
    | 37 ⇒ a = (RV32IM.execute_DIV_REM_pure b c .DRU).2
    | _ ⇒ True
```

noting that the right-hand sides are proven formalisations of the precise behaviour of the RISC-V specification. This formalisation reveals two important insights into the design of the ALU bus:

- only the ADD/SUB/AND/OR/XPR opcodes enforces the range of the operands, whereas for the other ALU opcodes the chip calling the ALU chip has to guarantee this instead;
- its modularity is extremely well-suited for formal verification, in that:
  - having the ALU bus relieves the framework of the need for embedding any ALU-related constraints into more complex airs that rely on ALU operations—all that is required is an ALU bus read;
  - when doing the verification, this bus (and specifically the `Spec` function, acts as a clean interface between the ALU airs and the other airs in that on the ALU side, we just have to prove the specification assuming that the air calling it satisfies the externals, and then for the air calling it we just have to prove that the externals hold and we can use the ALU specification.

2.3.3 The `Byte` bus

The `Byte` bus captures various bit- and range operations at the level of single bytes. Its structure and semantics are formalised as follows, noting that for the latter, and as for the ALU bus, we provide a specification function. Note that since this bus derives its meaning from the lookup tables that are constructed statically, we do not prove that these specifications hold (other than convincing ourselves by looking at the corresponding Rust code), but only assume it when the bus entries are read.

```
/-- Byte bus entry -/
structure ByteBusEntry where
  multiplicity    : F -- Lookup multiplicity
  byte_opcode     : F -- Byte opcode
  x_0 x_1 x_2 x_3 : F -- Bus arguments

/-- Byte bus pure specification -/
def ByteBus.Spec (byte_opcode x_0 x_1 x_2 x_3 : FKB) :=
  match byte_opcode with
  | 0 ⇒ x_0 = x_2 &&& x_3 ∧ x_0 < 256 ∧ x_1 = 0 ∧ x_2 < 256 ∧ x_3 < 256 -- bitwise AND
  | 1 ⇒ x_0 = x_2 ||| x_3 ∧ x_0 < 256 ∧ x_1 = 0 ∧ x_2 < 256 ∧ x_3 < 256 -- bitwise OR
  | 2 ⇒ x_0 = x_2 ^^^ x_3 ∧ x_0 < 256 ∧ x_1 = 0 ∧ x_2 < 256 ∧ x_3 < 256 -- bitwise XOR
  -- logical shift left with wrapping
  | 3 ⇒ x_0 = (x_2 <<< x_3) % 256 ∧ x_0 < 256 ∧ x_1 = 0 ∧ x_2 < 256 ∧ x_3 < 256
```

```
  -- logical shift right with carry
  | 4 ⇒ x_0 = x_2 >>> x_3 ∧ x_1 = x_2 - (x_0 <<< x_3) ∧ x_0 < 256 ∧ x_1 < 256 ∧ x_2 < 256 ∧ x_3 < 256
  -- unsigned less-than
  | 5 ⇒ (x_0 = if x_2 < x_3 then 1 else 0) ∧ x_1 = 0 ∧ x_2 < 256 ∧ x_3 < 256
  -- MSB
  | 6 ⇒ (x_0 = if x_2 < 128 then 0 else 1) ∧ x_0 < 256 ∧ x_1 = 0 ∧ x_2 < 256 ∧ x_3 < 256
  | 7 ⇒ x_0 = 0 ∧ x_1 = 0 ∧ x_2 < 256 ∧ x_3 < 256 -- 8-bit range
  | 8 ⇒ x_0 < 65536 ∧ x_1 = 0 ∧ x_2 = 0 ∧ x_3 = 0 -- 16-bit range
  | _ ⇒ True

/-- Byte bus entry instance -/
instance ByteBusEntryInstance
: BusEntry FKB (ByteBusEntry FKB) :=
{
  multiplicity := fun entry ⇒ entry.1,

  data_length := 5,
  data := fun ⟨_, byte_opcode, x_0, x_1, x_2, x_3⟩ ⇒ #v[byte_opcode, x_0, x_1, x_2, x_3]

  -- No axioms and externals
  axioms := fun _ ⇒ True
  externals := fun _ ⇒ True

  -- Well-formedness captured by pure specification
  wf_properties := fun ⟨ _, byte_opcode, x_0, x_1, x_2, x_3 ⟩ ⇒ ByteBus.Spec byte_opcode x_0 x_1 x_2 x_3

  ...
}
```

2.3.4 The State bus

The State bus captures the interaction with RISC-V machine registers and memory. The entry structure and semantics of the State bus are formalised as follows:

```
/-- State bus entry -/
structure StateBusEntry where
  multiplicity : F
  chunk : F           -- Timestamp (determined by chunk and clock)
  clk   : F
  ptr   : F           -- Address (register or memory)
  x_0 x_1 x_2 x_3 : F -- Read or written value

/-- Register bus entry instance -/
@[simp, grind]
instance StateBusEntryInstance
: BusEntry FKB (StateBusEntry FKB) :=
{
  multiplicity := fun entry ⇒ entry.1,

  data_length := 7,
  data := fun ⟨_, chunk, clk, ptr, x_0, x_1, x_2, x_3⟩ ⇒ #v[chunk, clk, ptr, x_0, x_1, x_2, x_3]

  -- No axioms and externals
  axioms := fun _ ⇒ True
  externals := fun _ ⇒ True

  -- Well-formedness properties : the value is split into bytes
  wf_properties :=
    fun ⟨ _, chunk, clk, ptr, x_0, x_1, x_2, x_3 ⟩ ⇒ x_0 < 256 ∧ x_1 < 256 ∧ x_2 < 256 ∧ x_3 < 256

  ...
}
```

Importantly, the way in which the separation between registers and memory is achieved (as per §1.2, finding 3), is

that the first 32 addresses are interpreted as registers and the others as memory addresses. This will be critical when formalising the effect of the buses on the RISC-V state, as described shortly.

2.3.5 The `Memory` bus

The `Memory` bus summarises all memory operations, that is, the loading and storing of values. Its entry structure and semantics are formalised as follows:

```
/-- Memory bus entry -/
structure MemoryBusEntry where
  multiplicity    : F -- Lookup multiplicity
  mem_opcode      : F -- Memory opcode
  a_0 a_1 a_2 a_3 : F -- Value read or written from given address
  b_0 b_1 b_2 b_3 : F -- Address to read or write from,
  c_0 c_1 c_2 c_3 : F -- computed as b + c
  -- Indicator whether or not the register written to is zero
  a_is_0 : F
  -- Opcode selectors
  is_lb is_lbu is_lh is_lhu is_lw is_sb is_sh is_sw : F

/-- Memory bus pure specification -/
def MemoryBus.Spec (mem_opcode a_0 a_1 a_2 a_3 b_0 b_1 b_2 b_3 c_0 c_1 c_2 c_3 : FKB) : Prop :=
  let a := U32.toBV #v[a_0, a_1, a_2, a_3]
  let b := U32.toBV #v[b_0, b_1, b_2, b_3]
  let c := U32.toBV #v[c_0, c_1, c_2, c_3]
  let addr := (b + c).toNat
  -- Read value is split into bytes
  (mem_opcode < 15 → a_0 < 256 ∧ a_1 < 256 ∧ a_2 < 256 ∧ a_3 < 256) ∧
  -- Address is in range
  addr < Pico_address_space_size_32 ∧
  -- Address is divisible by two for LH and LHU and SH
  (mem_opcode = 11 ∨ mem_opcode = 14 ∨ mem_opcode = 16 → 2 ∣ addr) ∧
  -- Address is divisible by four for LW and SW
  (mem_opcode = 12 ∨ mem_opcode = 17 → 4 ∣ addr) ∧
  -- Sign-extended load opcodes: LB (10) and LH (11)
  (mem_opcode = 10 → a = BitVec.extend (BitVec.ofNat 8 a_0.val) 32 true) ∧
  (mem_opcode = 11 → a = BitVec.extend ((BitVec.ofNat 8 a_1.val) ++ (BitVec.ofNat 8 a_0.val)) 32 true) ∧
  -- Non-sign-extended load opcodes: LBU (13) and LHU (14)
  (mem_opcode = 13 → a = BitVec.extend (BitVec.ofNat 8 a_0.val) 32 false) ∧
  (mem_opcode = 14 → a = BitVec.extend ((BitVec.ofNat 8 a_1.val) ++ (BitVec.ofNat 8 a_0.val)) 32 false)

/-- Memory bus entry instance -/
instance MemoryBusEntryInstance
: BusEntry FKB (MemoryBusEntry FKB) :=
{
    multiplicity := fun entry ⇒ entry.1,

    data_length := 22,
    data := fun ⟨_, mem_opcode, a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3, c_0, c_1, c_2, c_3, a_is_0,
                 is_lb, is_lbu, is_lh, is_lhu, is_lw, is_sb, is_sh, is_sw⟩ ⇒
              #v[mem_opcode, a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3, c_0, c_1, c_2, c_3, a_is_0,
                 is_lb, is_lbu, is_lh, is_lhu, is_lw, is_sb, is_sh, is_sw]

    -- No axioms
    axioms := fun _ ⇒ True

    -- Opcode and selectors are constrained externally
    externals := fun ⟨multiplicity, mem_opcode, a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3, c_0, c_1, c_2, c_3, _,
                      is_lb, is_lbu, is_lh, is_lhu, is_lw, is_sb, is_sh, is_sw⟩ ⇒
      ¬(multiplicity = 0) →
        mem_opcode ∈ Finset.Icc 10 17 ∧                 -- Memory opcodes are between 10 and 17
        b_0 < 256 ∧ b_1 < 256 ∧ b_2 < 256 ∧ b_3 < 256 ∧ -- `b` operand is split into bytes
        c_0 < 256 ∧ c_1 < 256 ∧ c_2 < 256 ∧ c_3 < 256 ∧ -- `c` operand is split into bytes
        -- Selectors for load opcodes
        (mem_opcode = 10 →
```

```
          ( is_lb, is_lbu, is_lh, is_lhu, is_lw, is_sb, is_sh, is_sw ) = ( 1, 0, 0, 0, 0, 0, 0, 0 )) ∧
        (mem_opcode = 11 →
          ( is_lb, is_lbu, is_lh, is_lhu, is_lw, is_sb, is_sh, is_sw ) = ( 0, 0, 1, 0, 0, 0, 0, 0 )) ∧
        (mem_opcode = 12 →
          ( is_lb, is_lbu, is_lh, is_lhu, is_lw, is_sb, is_sh, is_sw ) = ( 0, 0, 0, 0, 1, 0, 0, 0 )) ∧
        (mem_opcode = 13 →
          ( is_lb, is_lbu, is_lh, is_lhu, is_lw, is_sb, is_sh, is_sw ) = ( 0, 1, 0, 0, 0, 0, 0, 0 )) ∧
        (mem_opcode = 14 →
          ( is_lb, is_lbu, is_lh, is_lhu, is_lw, is_sb, is_sh, is_sw ) = ( 0, 0, 0, 1, 0, 0, 0, 0 )) ∧
        -- Selectors and byte-splitting of the written value for the store opcodes
        (mem_opcode = 15 →
          a_0 < 256 ∧ a_1 < 256 ∧ a_2 < 256 ∧ a_3 < 256 ∧
          ( is_lb, is_lbu, is_lh, is_lhu, is_lw, is_sb, is_sh, is_sw ) = ( 0, 0, 0, 0, 0, 1, 0, 0 )) ∧
        (mem_opcode = 16 →
          a_0 < 256 ∧ a_1 < 256 ∧ a_2 < 256 ∧ a_3 < 256 ∧
          ( is_lb, is_lbu, is_lh, is_lhu, is_lw, is_sb, is_sh, is_sw ) = ( 0, 0, 0, 0, 0, 0, 1, 0 )) ∧
        (mem_opcode = 17 →
          a_0 < 256 ∧ a_1 < 256 ∧ a_2 < 256 ∧ a_3 < 256 ∧
          ( is_lb, is_lbu, is_lh, is_lhu, is_lw, is_sb, is_sh, is_sw ) = ( 0, 0, 0, 0, 0, 0, 0, 1 ))

    -- Well-formedness captured by pure specification
    wf_properties := fun ⟨_, mem_opcode, a_0, a_1, a_2, a_3, b_0, b_1, b_2, b_3, c_0, c_1, c_2, c_3,
                          _, _, _, _, _, _, _, _, _⟩ ⇒
      MemoryBus.Spec mem_opcode a_0 a_1 a_2 a_3 b_0 b_1 b_2 b_3 c_0 c_1 c_2 c_3

    ...
}
```

When it comes to the `Memory` bus, there are a number of matters of note:

- the `MemoryBus.Spec` function captures only the pure part of the specification—it does not state whether an actual load or store happened or the address at which it happened;
- the air that writes to the memory bus must ensure that a loaded value is correctly split into bytes (well-formedness);
- the air that reads the memory bus must ensure that a to-be-stored value is correctly split into bytes (externals);
- the air that reads the memory bus must ensure that the selectors are binary and one-shot (externals);

2.4 Bus operations

The final step when it comes to working with the Pico buses is to formalise their effect on the RISC-V state. This is done by first defining a bus row, which is intended to capture all of the bus interactions of a single row of a given air:

```
structure BusRow where
  Program : List (ProgramBusEntry FKB)
  ALU : List (ALUBusEntry FKB)
  Byte : List (ByteBusEntry FKB)
  State : List (StateBusEntry FKB)
  Memory : List (MemoryBusEntry FKB)
```

and then specifying how a bus row interacts with the RISC-V state:

```
def bus_operations (row : BusRow) : List Operation :=
  -- The program bus reads the value of the current program counter
  let program : List Operation :=
    List.map (fun program_row ⇒ .ReadPC (BitVec.ofNatLT (w := 32) program_row.pc (by omega))) row.Program
  -- The state bus reads and writes to registers and memory
  let state : List Operation :=
    List.filterMap (
      fun state_row ⇒
        -- Value that is read/written
        let value := U32.toBV #v[state_row.x_0, state_row.x_1, state_row.x_2, state_row.x_3]
        -- When address is less than 32, we are working with registers
```

```
      if _ : state_row.ptr < 32 then
        (let register : regidx :=
           .Regidx (BitVec.ofNatLT (w := if false = true then 4 else 5) state_row.ptr.val (by simpa))
        -- Negative multiplicity means "read"
        if (state_row.multiplicity = -1) then .some (.ReadReg register value)
        -- Positive multiplicity means "write"
        else if (state_row.multiplicity = 1) then .some (.WriteReg register value)
        -- Zero multiplicity has no effect
        else .none)
      -- Otherwise, we are working with memory
      else
        -- Negative multiplicity means "read", and we always read the entire value
        if (state_row.multiplicity = -1) then .some (.ReadMem state_row.ptr.val .Word value)
        -- Positive multiplicity means "write", and we always write the entire value
        else if (state_row.multiplicity = 1) then .some (.WriteMem state_row.ptr.val .Word value)
        -- Zero multiplicity has no effect
        else .none
  ) row.State
-- The memory bus summarises fine-grained memory operations
let memory : List Operation :=
  List.filterMap (
    fun (memory_row : MemoryBusEntry FKB) ⇒
      -- Compute width
      let width : Option MemoryWidth :=
        if (memory_row.is_lw = 1 ∨ memory_row.is_sw = 1) then .some .Word
        else if (memory_row.is_lh = 1 ∨ memory_row.is_lhu = 1 ∨ memory_row.is_sh = 1) then .some .Half
        else if (memory_row.is_lb = 1 ∨ memory_row.is_lbu = 1 ∨ memory_row.is_sb = 1) then .some .Byte
        else .none
      match width with
      | .none ⇒ .none
      | .some width ⇒
        -- Compute address
        let addr :=
          (U32.toBV #v[memory_row.b_0, memory_row.b_1, memory_row.b_2, memory_row.b_3]
             + U32.toBV #v[memory_row.c_0, memory_row.c_1, memory_row.c_2, memory_row.c_3]).toNat
        let a_0, a_1, a_2, a_3 := memory_row.a_0, memory_row.a_1, memory_row.a_2, memory_row.a_3
        -- Store indicator
        let is_store : Bool := memory_row.is_sb == 1 ∨ memory_row.is_sh == 1 ∨ memory_row.is_sw == 1
        match is_store with
        | true ⇒
          -- Storing, fine-grained
          match width with
          | .Byte ⇒ .some (.WriteMem addr .Byte (BitVec.ofNat 8 a_0.val))
          | .Half ⇒ .some (.WriteMem addr .Half (BitVec.ofNat 8 a_1.val ++ BitVec.ofNat 8 a_0.val))
          | .Word ⇒ .some (.WriteMem addr .Word (U32.toBV #v[a_0, a_1, a_2, a_3]))
        | false ⇒
          -- Loading, fine-grained
          match width with
          | .Byte ⇒ .some (.ReadMem addr .Byte (BitVec.ofNat 8 a_0.val))
          | .Half ⇒ .some (.ReadMem addr .Half (BitVec.ofNat 8 a_1.val ++ BitVec.ofNat 8 a_0.val))
          | .Word ⇒ .some (.ReadMem addr .Word (U32.toBV #v[a_0, a_1, a_2, a_3]))
  ) row.Memory
program ++ state ++ memory
```

2.5 Opcode verification workflow

For every verified opcode verified, we perform the following workflow in Lean, noting that there are parts of the workflow that are re-used for multiple opcodes, as Pico chips almost never implement a single opcode:

1. implement the transpilation of the opcode in our formalisation of the Pico transpiler;
2. formalise the air that implements the opcode;
3. extract the constraints and bus contents of the air in question;

4. transform the extracted constraints and bus contents into human-readable form;
5. prove that the RISC-V operation effect of the opcode (cf. §2.1) is equivalent to the effect of its bus operations (§2.4), effectively proving overall correctness of the opcode implementation.

The main 45 correctness theorems, one per each analysed opcode, can all be found in the Equivalence.lean file.

As an example, we give the annotated correctness theorem for the ADD opcode:

```
theorem equivalence_ADD
  -- Let `row` be a non-padding, in-range row of the CPU air `air`
  (air : Valid_CpuAir FKB ExtF)
  (row : ℕ)
  (h_row : row ≤ air.last_row)
  (h_is_real : air.cols.is_real row 0 = 1)
  -- and let hold the axioms and well-formedness properties of the corresponding CPU bus row
  (h_axioms : bus_row_axioms (bus_row air row))
  (h_wf : bus_row_wf_asm (bus_row air row))
  -- as well as the appropriate constraints.
  (h_constraints : CpuAir.Extraction.allHold air row)
  -- Further, assume that we are executing the `ADD` opcode
  (h_opcode : air.cols.instruction.opcode = 0 ∧ air.cols.opcode_selector.imm_c row 0 = 0)
  -- and that the hypotheses of the CPU bus row operations hold.
  (h_bus_hypotheses : (operation_list_effect (RV32IM.bus_operations (bus_row air row)) state).hyps)
:
  -- Then, executing the `ADD` instruction is equivalent
  -- to the effect of the appropriate CPU bus row operations.
  let reg0 : regidx := .Regidx (air.cols.instruction.a_0 row 0).val
  let reg1 : regidx := .Regidx (air.cols.instruction.b_0 row 0).val
  let reg2 : regidx := .Regidx (air.cols.instruction.c_0 row 0).val
  execute_instruction (instruction.RTYPE (reg2, reg1, reg0, rop.ADD)) state =
  (operation_list_effect (CPU_bus_operations air row) state).state
```

3 Disclaimer

This Report is based on the scope of materials and documentation provided by you to Nethermind in order that Nethermind could conduct the tasks outlined in the Executive Summary. The content of this Report is intended to provide an analysis of the formal verification process undertaken, and the results obtained, based on the information and data available to Nethermind up to the time of completion. The formal verification process conducted by the team of engineers was based on the specific parameters, methodologies, and data available at the time of the analysis. Any changes in the underlying parameters, methodologies, or data may lead to different results.

Nethermind has provided the review and this Report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to any other areas outside the scope of work agreed with you in our Agreement. This Report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this Report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset.

To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this Report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the Report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.